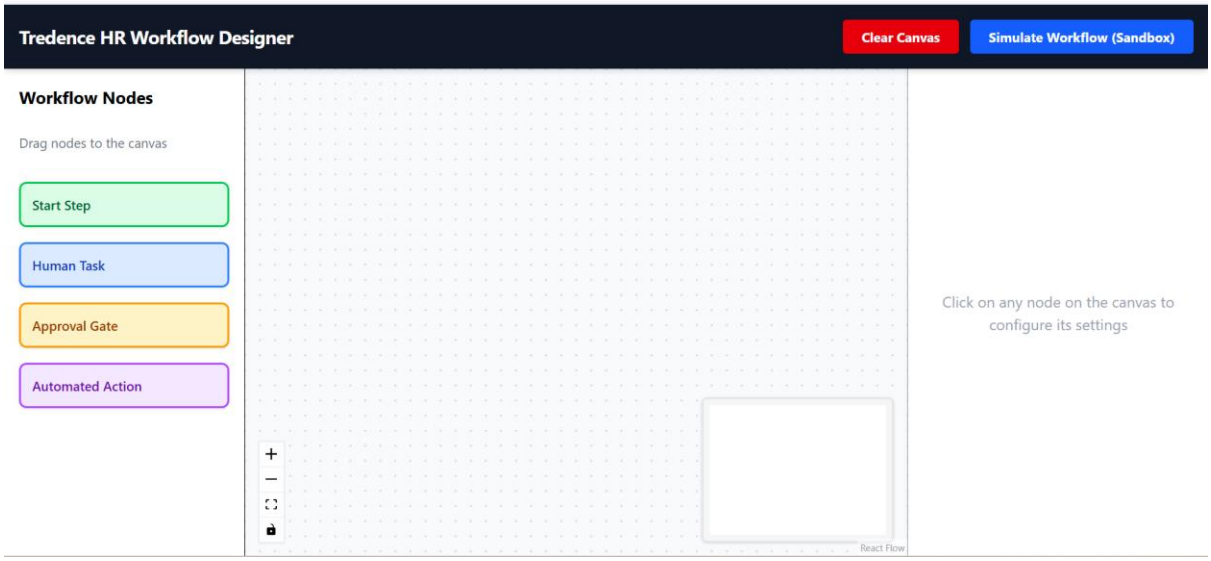
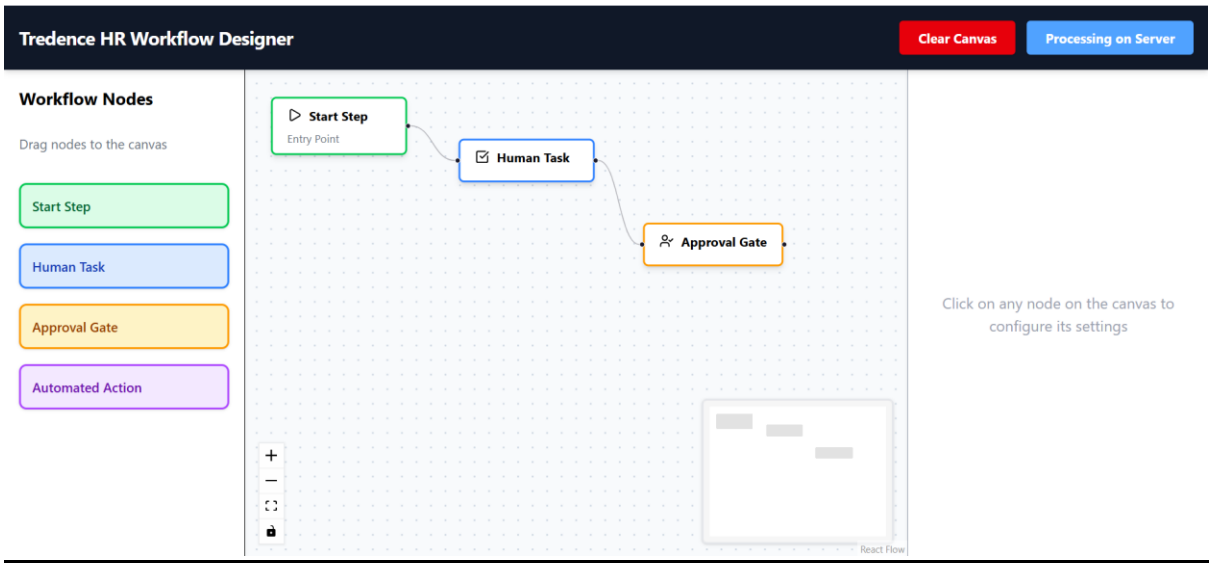
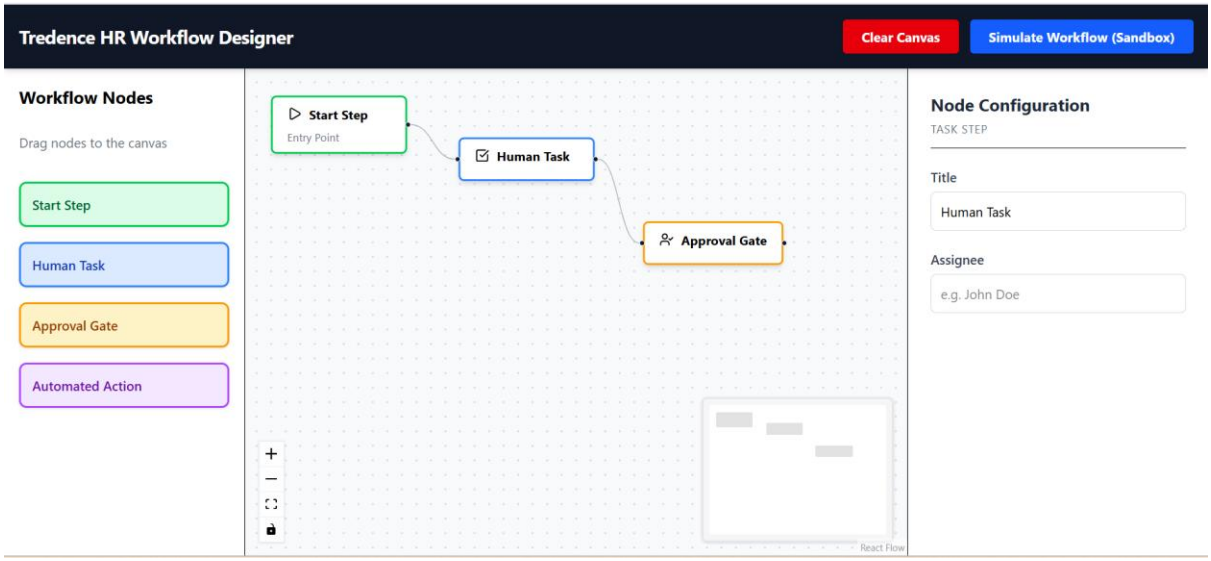
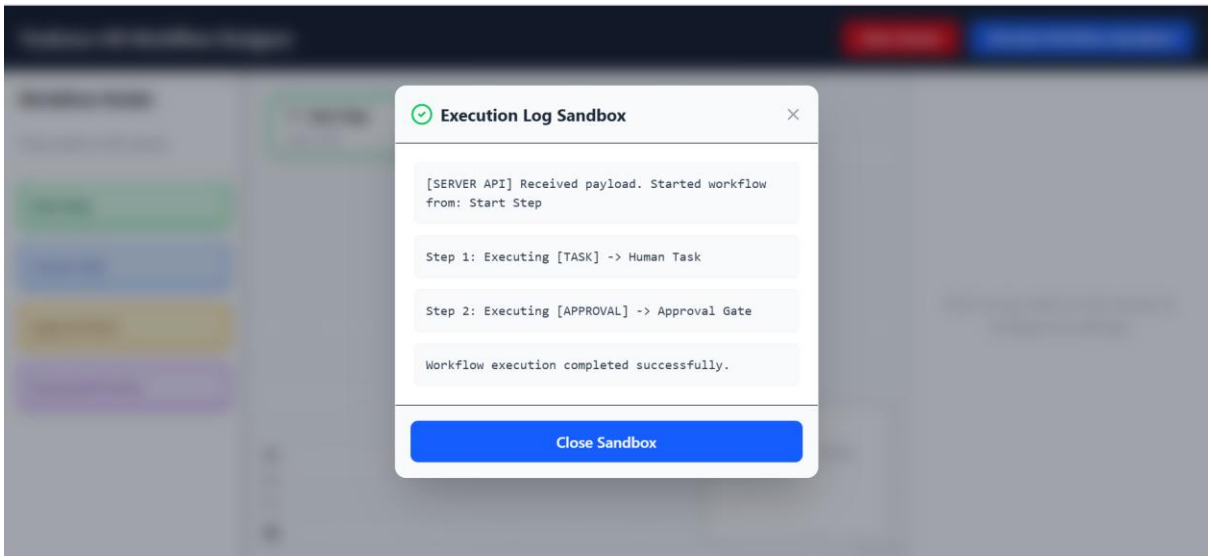
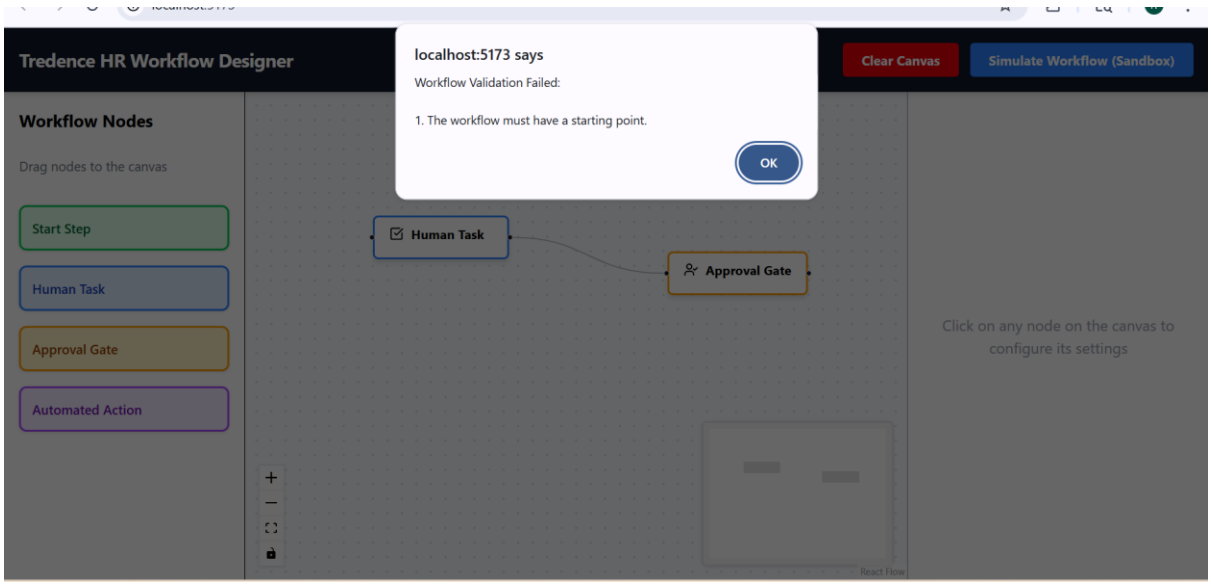
Tredence HR Workflow Designer - Case Study Submission - Rhythm Garg

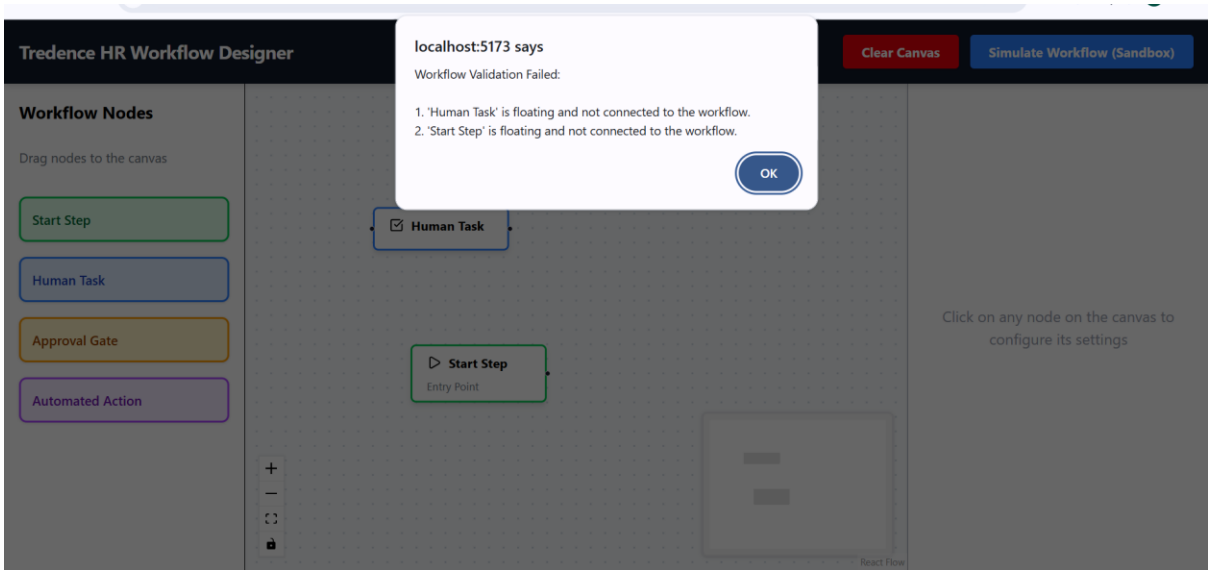


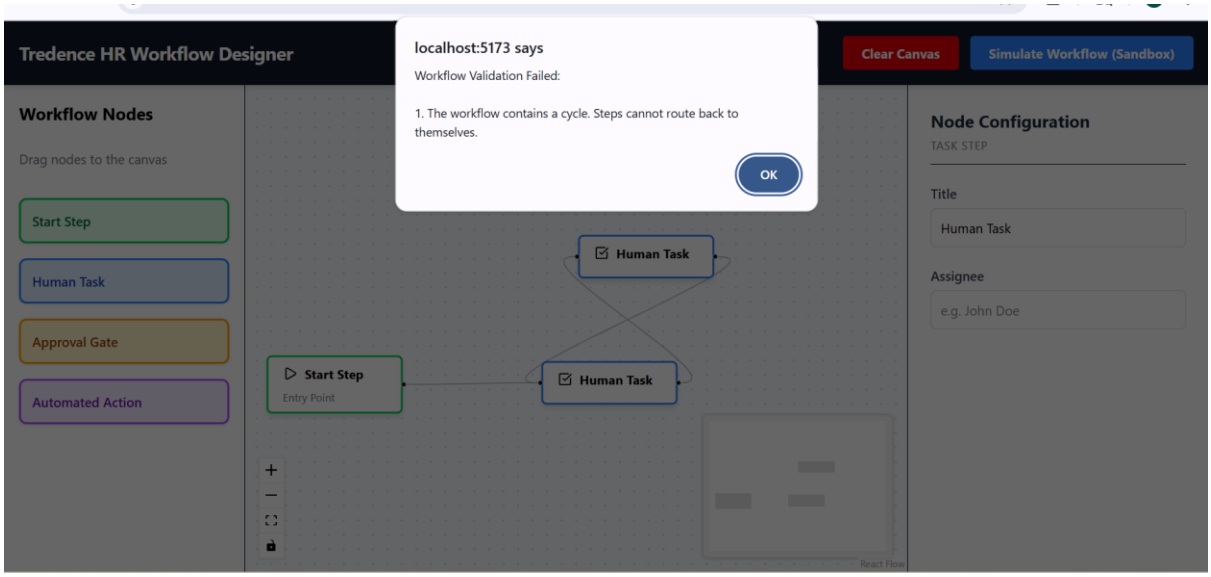



Tredence HR Workflow Designer
Clear Canvas
Simulate Workflow (Sandbox)
Workflow Nodes
Drag nodes to the canvas
Start Step
Human Task
Approval Gate
Automated Action
Start Step
Entry Point
Human Task
Approval Gate
Node Configuration
TASK STEP
Title
Human Task
Assignee
e.g. John Doe
React Flow


Execution Log Sandbox
[SERVER API] Received payload. Started workflow from: Start Step
Step 1: Executing [TASK] -> Human Task
Step 2: Executing [APPROVAL] -> Approval Gate
Workflow execution completed successfully.
Close Sandbox


Tredence HR Workflow Designer
localhost:5173 says
Workflow Validation Failed:
1. The workflow must have a starting point.
OK
Clear Canvas
Simulate Workflow (Sandbox)
Workflow Nodes
Drag nodes to the canvas
Start Step
Human Task
Approval Gate
Automated Action
Human Task
Approval Gate
Click on any node on the canvas to configure its settings
React Flow





Architecture & Design Decisions

Given the 24-hour timebox, my primary focus was on establishing a clean, scalable React architecture rather than pixel-perfect styling.

- **State Management**: Utilized React Flow's native hooks (useNodesState, useEdgesState) combined with lifting state to the App.jsx container component. This "Smart Container / Dumb Component" pattern keeps the UI rendering components (CanvasArea, ConfigPanel, Sidebar) stateless and highly reusable.
- **Mock API Abstraction**: Instead of executing synchronous logic on the frontend, I abstracted the simulation into an asynchronous service layer (src/services/api.js). This

utilizes Promises and setTimeout to mimic network latency, allowing the UI to accurately demonstrate loading states and asynchronous error handling.

- **Graph Validation**: Before sending the payload to the server, the application runs a local validation utility (src/utils/workflowValidator.js). I implemented a Depth-First Search (DFS) algorithm to detect cycles (infinite loops) and logic to identify floating/orphan nodes, ensuring only valid directed acyclic graphs (DAGs) are processed.
- **Styling**: Leveraged Tailwind CSS (v4) for rapid, utility-first styling, ensuring a clean and responsive layout without writing custom CSS files.

Folder Structure

```
src/
├── components/        # Pure, presentational UI components
│   ├── CanvasArea.jsx   # React Flow canvas wrapper
│   ├── ConfigPanel.jsx  # Dynamic right-hand properties form
│   ├── CustomNodes.jsx  # Visual definitions for the 4 node types
│   └── Sidebar.jsx      # Drag-and-drop source panel
├── services/
│   └── api.js         # Asynchronous mock API layer
├── utils/
│   └── workflowValidator.js # DFS cycle detection and rule validation
├── App.jsx            # Main state container and layout logic
└── main.jsx           # React entry point
```

Future Enhancements

If given more time, I would implement the following features:

1. **Zustand or Redux**: Move the node/edge state out of App.jsx into a global store to prevent unnecessary re-renders of the entire layout when a single node property updates.
2. **Real Backend Integration**: Swap the mock API service with a lightweight Python FastAPI backend to handle the graph traversal and payload execution natively.

3. **JSON Export/Import**: Add utility functions to stringify the canvas state into a downloadable .json file, and a file-reader input to hydrate the canvas from a saved configuration.
4. **Enhanced Node Feedback**: Visually highlight the specific nodes on the canvas that fail validation (e.g., turning a node border red if it causes an infinite loop).

For complete code :- GitHub Project Link